



APRENDIZAJE SUPERVISADO 2025

Docentes:

Dra. Ing. Karim Nemer Pelliza

Dr. Ing. Diego González Dondo



CONTENIDO DE LA PRIMERA CLASE

- ✓ Introducción al ML
- ✓ Etapas en la aplicación del ML
- ✓ Repaso:
 - ✓ Regresión Lineal y Polinomial,
 - ✓ Regresión Logística
 - ✓ Perceptrón.
- ✓ Support Vector Machines.
- ✓ SVC/SVR.
 - ✓ Datos no linealmente separables.
 - ✓ Función de costo.
- ✓ Redes neuronales multicapa
- ✓ Naive Bayes
- ✓ Repaso:
 - ✓ Árboles de decisión
- ✓ Ensemble learning.
 - ✓ Random Forest,
 - ✓ Bagging, Boosting y Voting.
- ✓ Sistemas de recomendación.
 - ✓ Filtrado colaborativo.
- ✓ Prácticas



PRIMERA CLASE



INTRODUCCIÓN AL MACHINE LEARNING

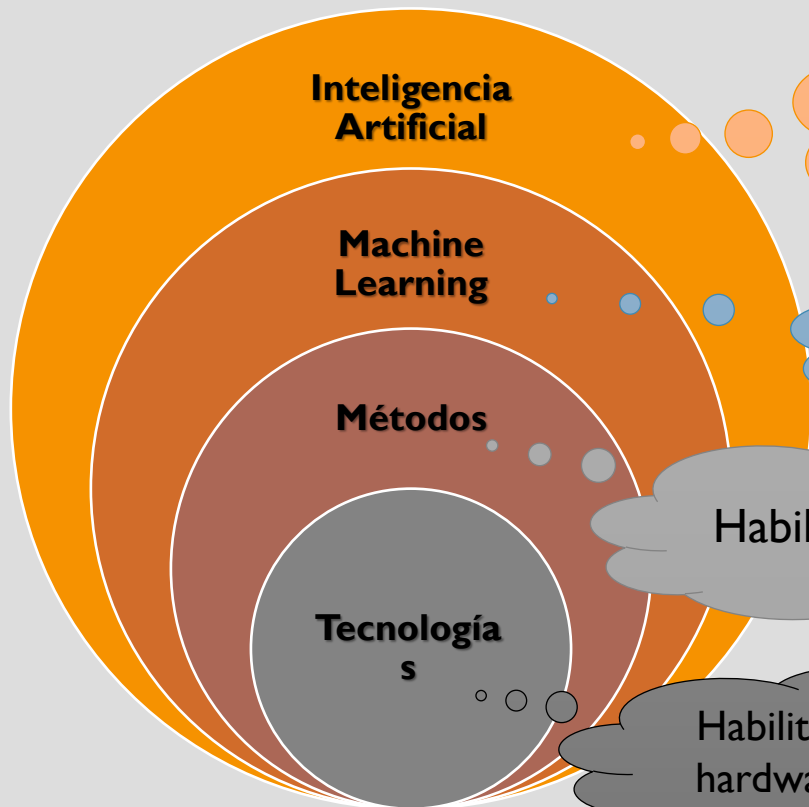
A.L Samuel

El aprendizaje automático es el proceso que le da a las computadoras la habilidad de aprender sin ser explícitamente programadas.

E.T.M Mitchell

Se dice que un programa de computación aprende de la experiencia E con respecto a una tarea T y alguna medida de rendimiento P , si es que el rendimiento en T , medido por P , mejora con la experiencia

¿CÓMO SE RELACIONAN LA IA Y EL ML?



Capacidad de
censar, razonar,
relacionar y
aprender

Habilidad de
aprender

Habilidad de razonar

Habilitación física de
hardware y software

- Visión por computadora
- Procesamiento del lenguaje natural
- Reconocimiento de voz
- Robótica y movimiento
- Planificación y optimización
- Simulación de conocimiento

- Aprendizaje supervisado,
- Aprendizaje no supervisado
- A. por refuerzo

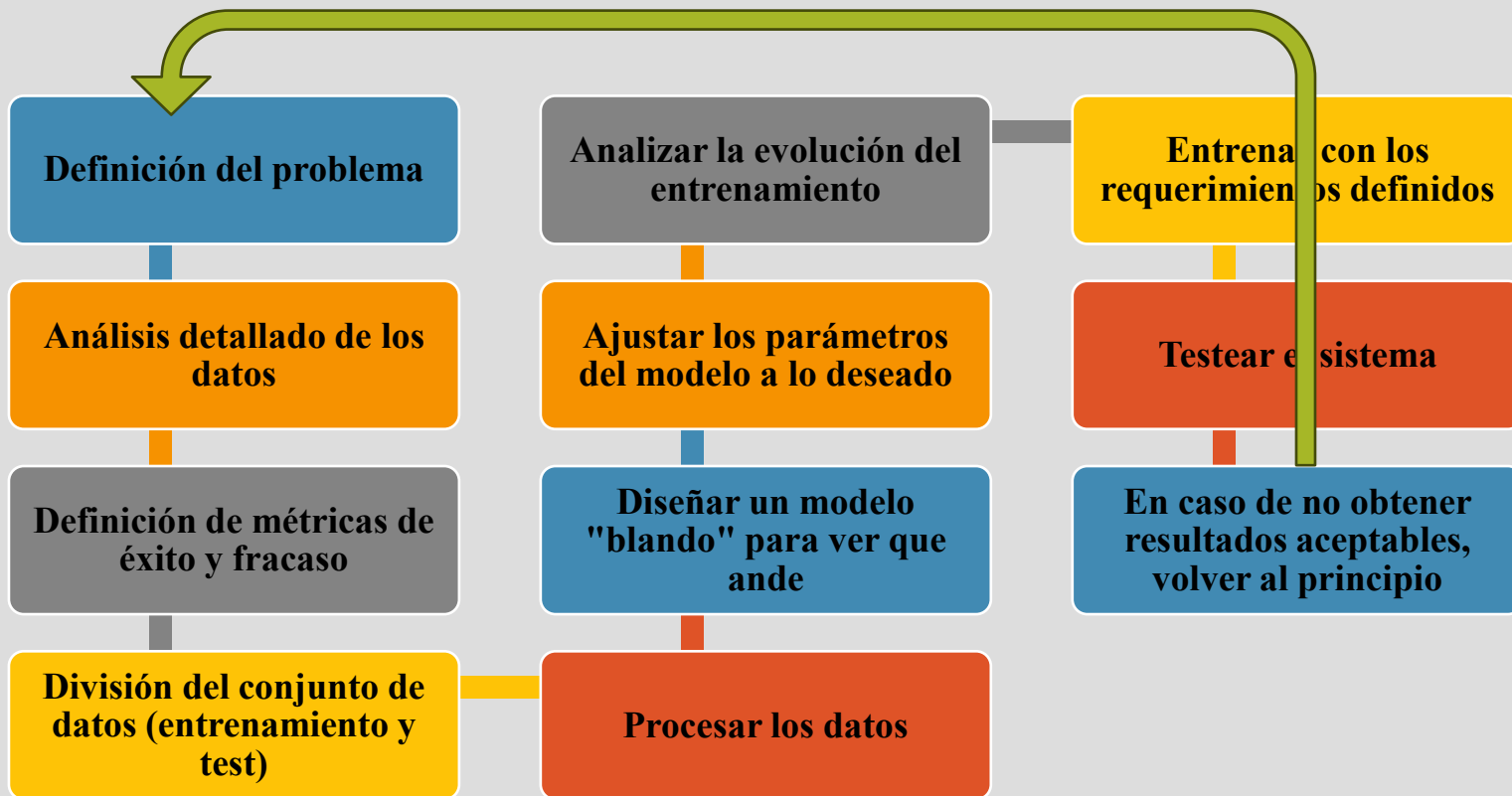
- Regresión
- Árboles de decisión
- ANN
- SVM
- Ensemble Learning



PRINCIPALES USOS DEL APRENDIZAJE SUPERVISADO



ETAPAS EN LA APLICACIÓN DEL APRENDIZAJE SUPERVISADO

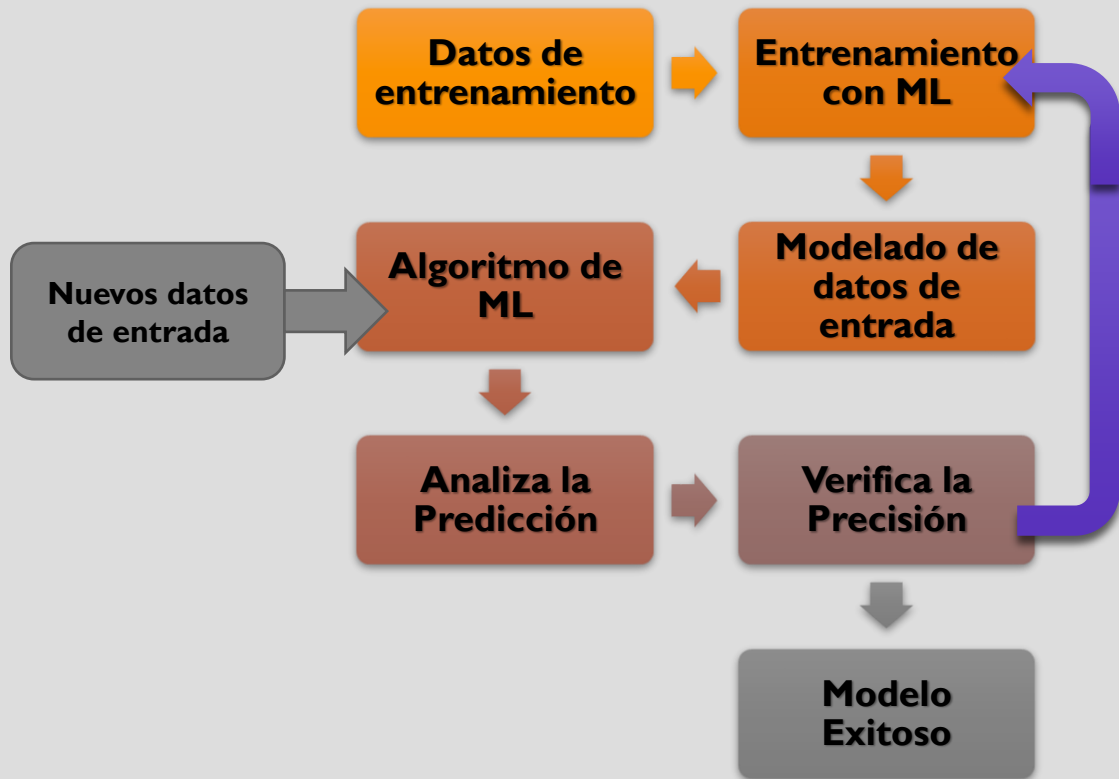




DESCRIPCIÓN DEL PROBLEMA: APRENDIZAJE SUPERVISADO

Datos: Se dispone de un conjunto de registros (o ejemplos, o instancias) descritos por n atributos: $A_1, A_2, \dots, A_n$ y cada instancia está anotada con una etiqueta, pudiendo ser una clase o un valor numérico.

Objetivo: Aprender un modelo (o función) a partir de los datos, buscando predecir sus etiquetas a partir de los atributos. Este modelo puede ser utilizado para predecir las etiquetas de nuevos registros sin anotar.





APRENDIZAJE SUPERVISADO

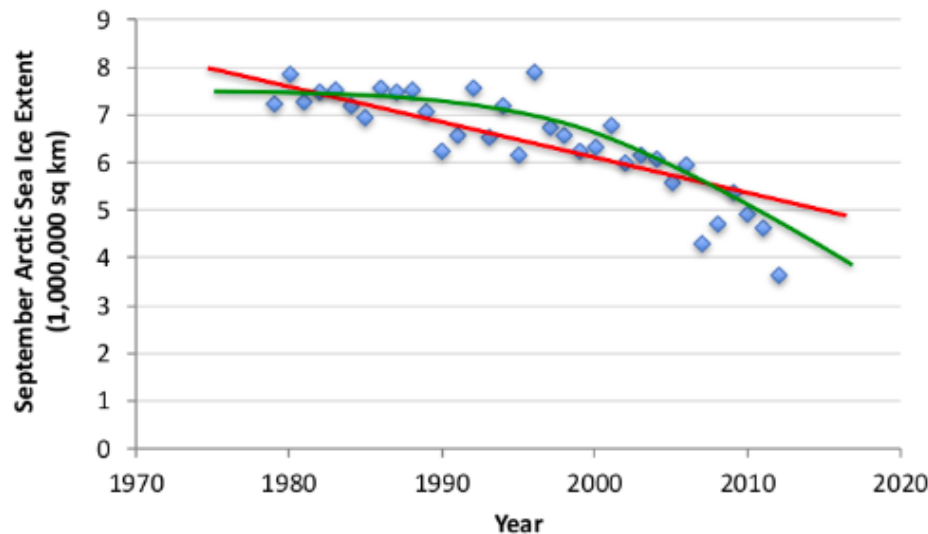


REGRESIÓN

Dados $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$

Aprender una $f(x)$ que permita predecir y a partir de x

Si $y \in \mathbb{R}^n$: Es un problema de **regresión**.





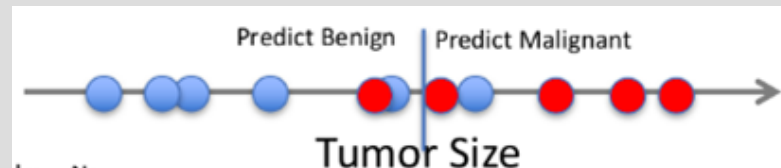
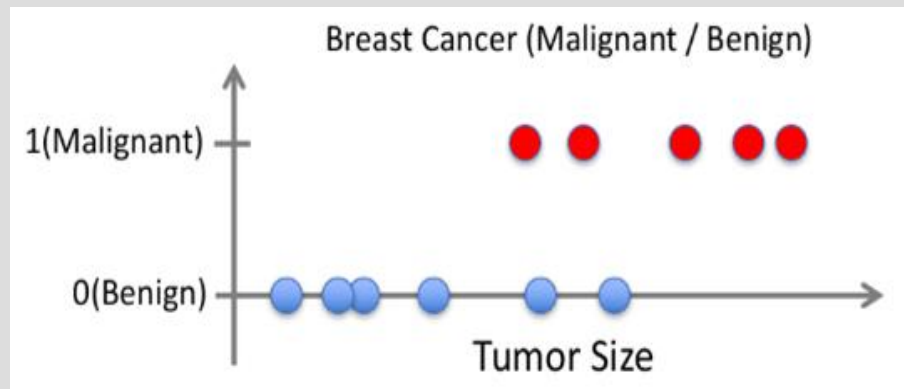
CLASIFICACIÓN

Dados $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$



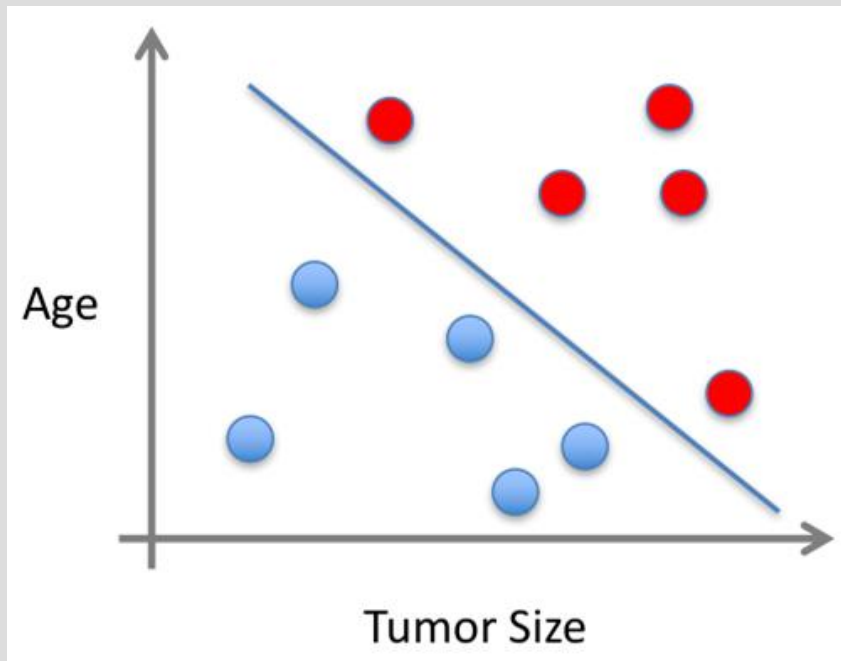
Aprender una $f(x)$ que permita
predecir y a partir de x

Si y es categórica: Es un
problema de **clasificación**.





APRENDIZAJE SUPERVISADO



La variable x puede ser multidimensional.

Cada dimensión corresponde a un atributo:

Edad del paciente

Tamaño del tumor

Uniformidad en la forma
de la célula

La regresión busca “acercar” los datos a una
función (lineal, polinomial, etc.)

La clasificación busca separar los datos
mediante ciertos “bordes”.



ELECCIÓN DE *HIPERPARÁMETROS*





REGRESIÓN LINEAL Y POLINOMIAL



REGRESIÓN LINEAL

Busca ajustar los datos de entrenamiento mediante una función que sea un hiperplano.

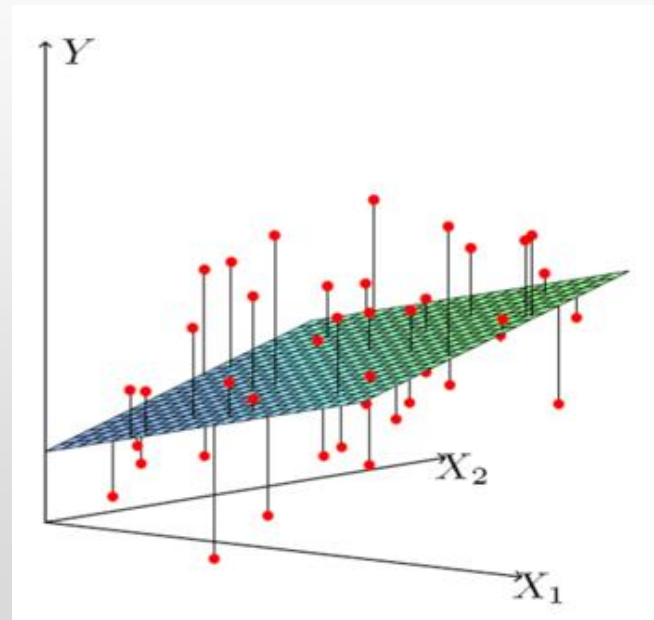
$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_d x_d = \sum_{j=0}^d \theta_j x_j$$

Los valores θ son los pesos de los atributos o *features*.

Se entrena minimizando la suma del error cuadrático.

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n (h_{\theta}(x^i) - y^i)^2$$

Se resuelve mediante $\min_{\theta} J(\theta)$





REGRESIÓN POLINOMIAL

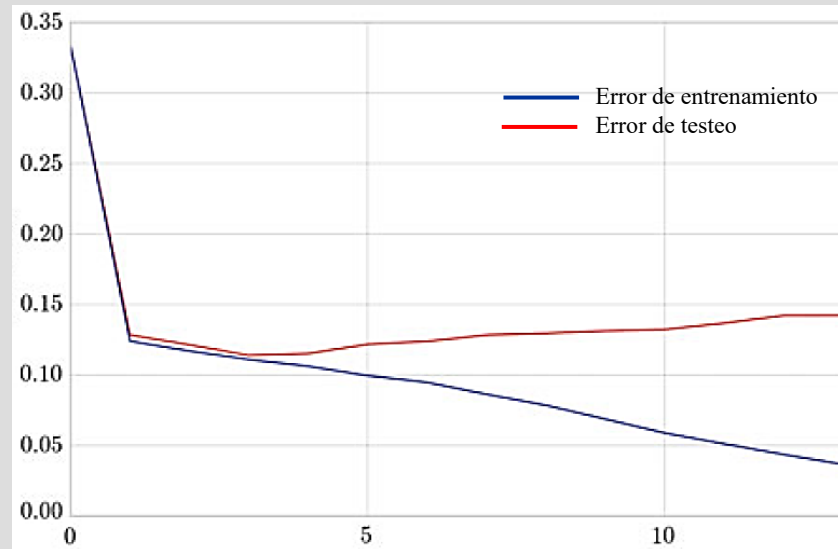
Busca ajustar los datos de entrenamiento mediante una función polinomial:

$$y(x, W) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M = \sum_{j=0}^M w_j x^j$$



Mientras más alto el grado del polinomio, más se ajusta a los datos

(pero se vuelve más complejo y tiende a sobreajustar).





DEMO TIME
(DEMO_1_LINEAR_REGRE
SSION)



REGRESIÓN LOGÍSTICA



REGRESIÓN LOGÍSTICA

Usa un enfoque probabilístico.

$h_{\theta}(x)$ debería devolver $p(y = 1|x; \theta)$

Como $0 \leq h_{\theta}(x) \leq 1$

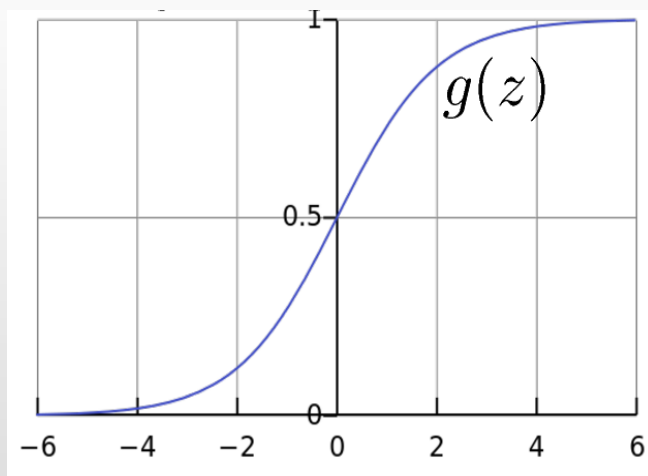


Modelo de regresión logística

$$h_{\theta}(x) = g(\theta^T x)$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

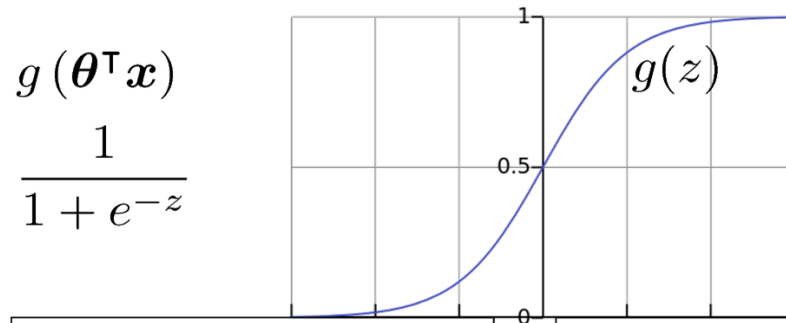




REGRESIÓN LOGÍSTICA

$$h_{\theta}(x) = g(\theta^T x)$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

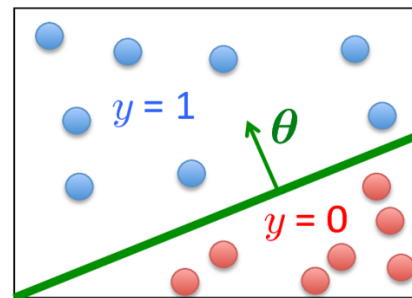


$\theta^T x$ debería tener valores **negativos** grandes para instancias negativas y valores **positivos** grandes para instancias positivas.

Definir un umbral y...

Predecir : $y = 1$ si $h_{\theta}(x) \geq 0.5$

Predecir : $y = 0$ si $h_{\theta}(x) < 0.5$





REGRESIÓN LOGÍSTICA: FUNCIÓN DE COSTO

$$J(\theta) = - \sum_{i=1}^n \left[y^i \log h_{\theta}(x^i) + (1 - y^i) \log (1 - h_{\theta}(x^i)) \right]$$



El costo de una sola instancia de los datos se define como:

$$cost(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{si } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{si } y = 0 \end{cases}$$



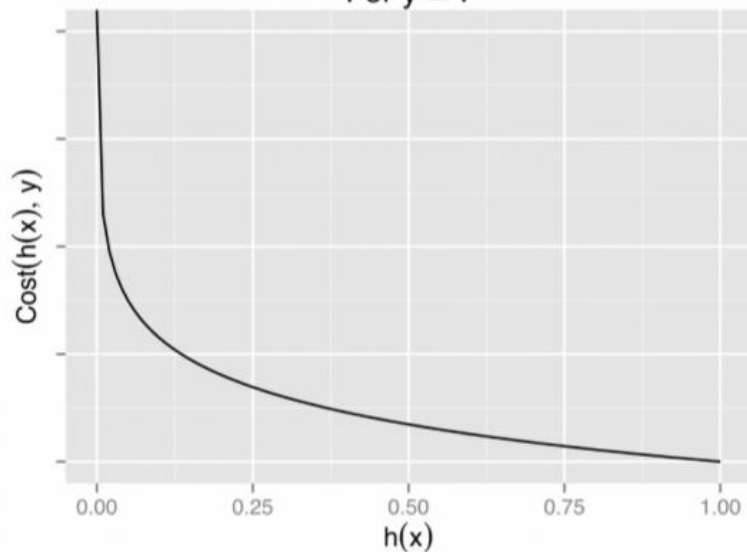
Reescribimos la función de costo como:

$$J(\theta) = - \sum_{i=1}^n [cost(h_{\theta}(x^i), y^i)]$$

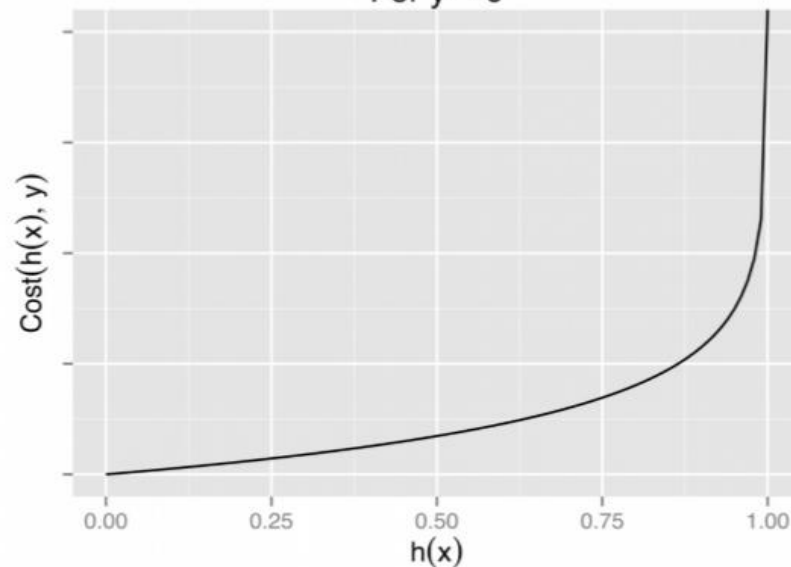


COMPORTAMIENTO DE LA FUNCIÓN DE COSTO

For $y = 1$



For $y = 0$





**DEMO TIME
(DEMO_2_LOGISTIC_REG
RESSION)**



ALGORITMO DEL PERCEPTRÓN



EL ALGORITMO DEL "PERCEPTRÓN"

Propuesto por Frank Rosenblatt en 1958

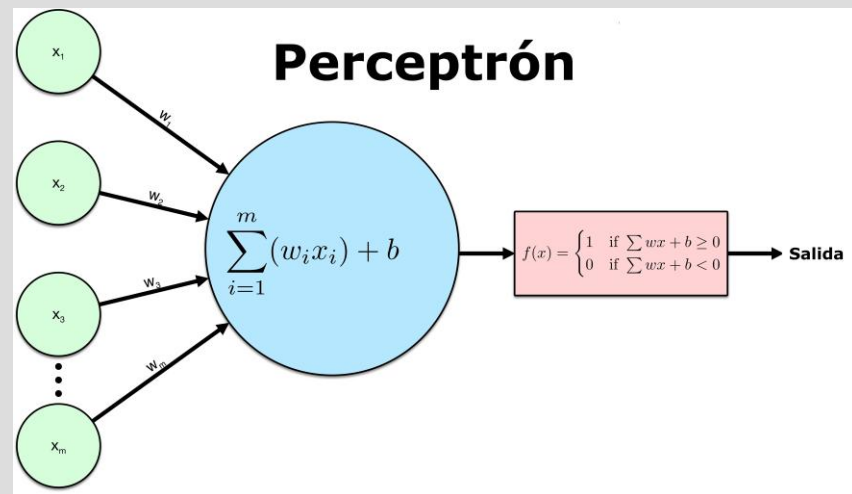
El objetivo es encontrar un hiperplano de separación

Sólo encuentra la solución si los datos son linealmente separables

Es un algoritmo online (procesa un ejemplo a la vez)

Es la red neuronal más simple

Tiene una capa de entrada y un único nodo de salida.



EL ALGORITMO DEL "PERCEPTRÓN"

Entrada

- Conjunto de entrenamiento
- Tasa de aprendizaje



Algoritmo

- Inicializar los pesos
- $\omega^0 \in \mathbb{R}$



Proceso

- Para cada ejemplo (x_i, y_i) predecir $y_i^t = \text{signo}(\omega^t x_i + \omega_0)$



Comparo salidas real y obtenida

- Si $y_i^t \neq y_i \therefore \omega^{t+1} \leftarrow \omega^t + r(y_i x_i)$



Sólo se actualiza cuando se comete un error



Si se han actualizado algunos pesos se vuelve a "Proceso", si no, se finaliza



**DEMO TIME
(DEMO_3_PERCEPTRO
N)**



Support Vector Machines



SVM: FRONTERAS DE DECISIÓN EN CLASIFICACIÓN

Un clasificador busca separar los datos de una y otra clase de la mejor manera.

Esta separación se da mediante una frontera de decisión.

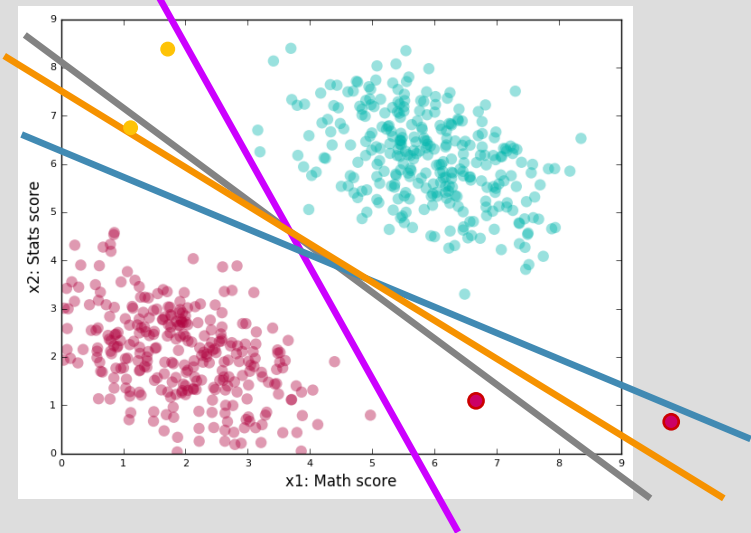
¿Qué determina que tan “buena” es una frontera de decisión?

Cualquiera de las líneas separa los datos correctamente.

Buscamos una línea que capture el patrón general entre los datos.

La línea fucsia tiene menos margen entre ella y ambos clústeres de datos.

La línea azul se encuentra bien a la mitad de ambos clústeres.





SUPPORT VECTOR MACHINES

Es un algoritmo que busca separar los datos mediante la mejor frontera de decisión.

Esta frontera de decisión es conocida como **hiperplano**.



En este caso, “*mejor*” se refiere a aquella que esté lo más separada posible de los puntos más cercanos a ella.

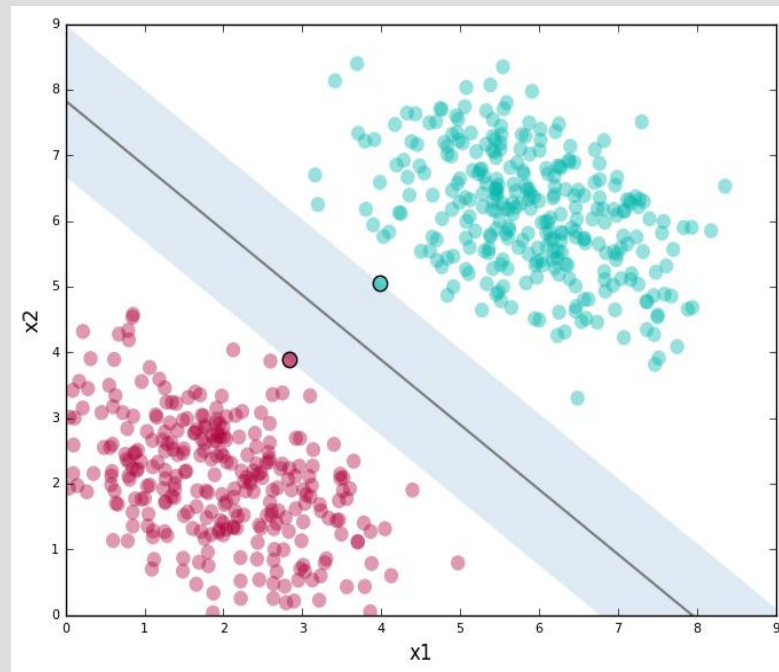
Estos puntos son conocidos como **vectores de soporte**, y el espacio entre ellos y el hiperplano se conoce como **margen**.



En términos más técnicos, un algoritmo de SVM encuentra el hiperplano que devuelva el mayor margen entre sí mismo y los **vectores de soporte**.



Este tipo de clasificador a veces es conocido como “clasificador por márgenes” (margin classifier).



[Link](#)



SVM: FUNCIÓN DE COSTO Y A OPTIMIZAR

Los SVM utilizan una función de costo conocida como *Hinge loss*.

A diferencia de regresión logística, los datos se anotan con $\{-1, 1\}$ de acuerdo al valor de la etiqueta.

La función de costo de Hinge se define como:

$$c(x, y, f(x)) = \max(0, 1 - y * f(x))$$

Donde el costo es 0 si el valor real y el predicho tienen el mismo signo y están dentro del margen de error (por lo general 1).

La función que buscamos minimizar es la siguiente:

$$\min_{\omega} \sum_{i=1}^n \max(0, 1 - y_i \langle x_i, \omega \rangle) + \lambda \|\omega\|^2$$

Dónde $\lambda \|\omega\|^2$ es el parámetro de regularización.



SVM: GRADIENTES

Tenemos dos factores en la función de costo que hay que derivar:

$$\frac{\delta}{\delta \omega_k} = \lambda \|\omega\|^2 = 2\lambda \omega_k$$

$$\frac{\delta}{\delta \omega_k} \max(0; 1 - y_i \langle x_i, \omega \rangle) = \begin{cases} 0 & \text{si } y_i \langle x_i, \omega \rangle \geq 1 \\ -y_i x_{ik} & \text{c. c.} \end{cases}$$



Al actualizar los pesos, de acuerdo al signo de la predicción, tendremos para el caso donde el signo sea el mismo:

$$\omega = \omega - \alpha(2\lambda\omega)$$



Mientras que cuando el signo entre la predicción y el valor real es diferente:

$$\omega = \omega - \alpha(y_i x_i - 2\lambda\omega)$$



SVM CON OUTLIERS

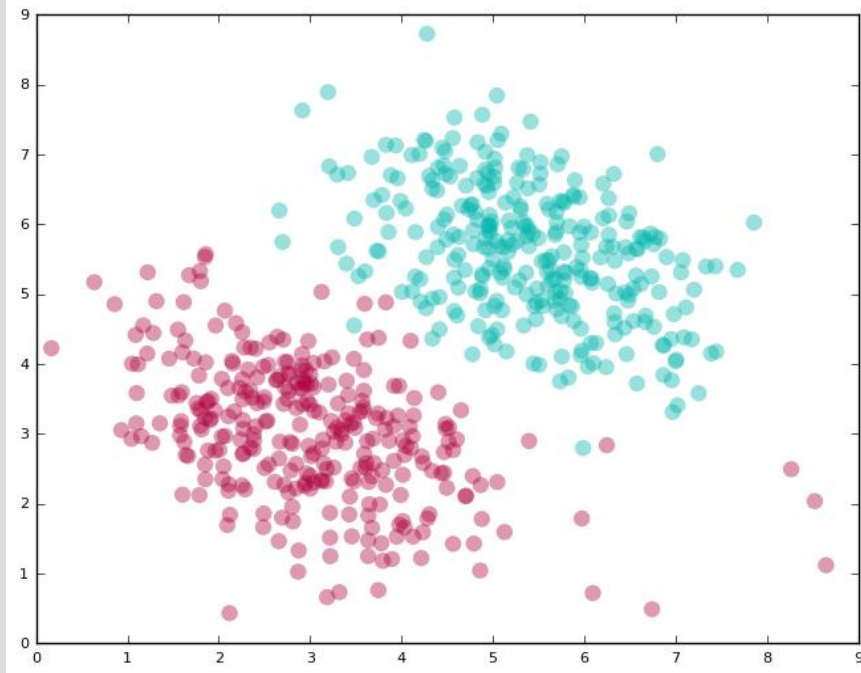
SVM: OUTLIERS

La mayoría de los casos, los datos no son linealmente separables.

En algunos casos, existen outliers.

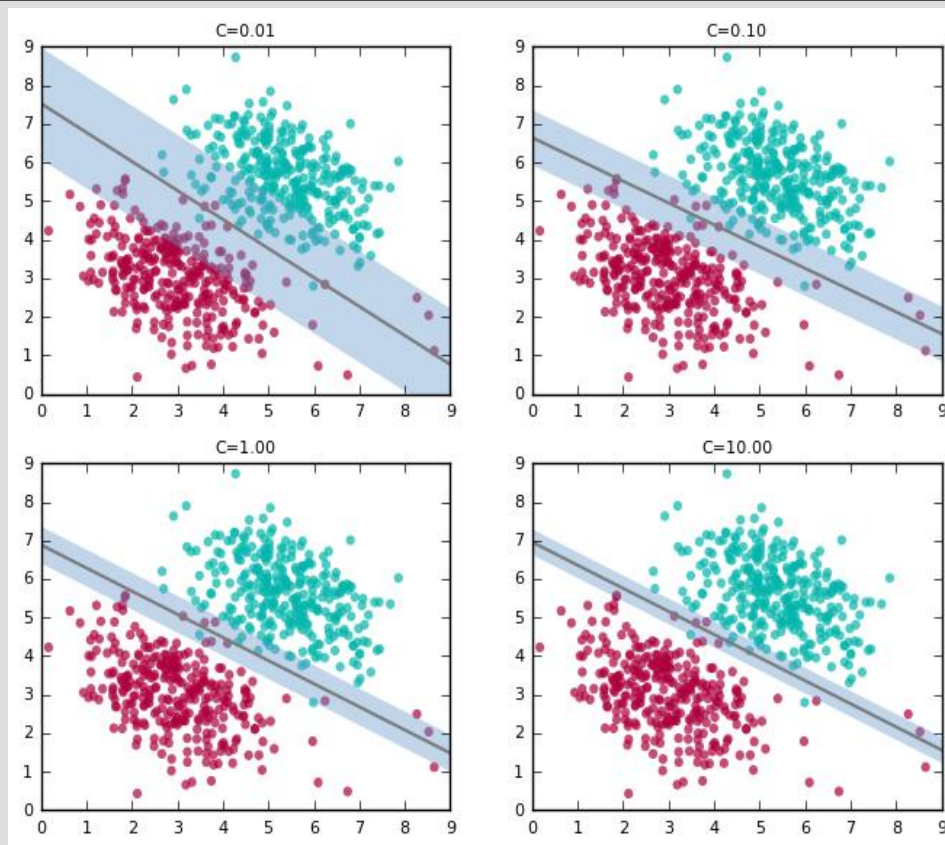
Hay un parámetro que define qué tan tolerante puede ser SVM sobre la clasificación incorrecta de datos.

El “*parámetro C*”, define un *tradeoff* entre clasificar mejor los datos de entrenamiento y tener una mejor “*separación*” (un margen más amplio).





SVM: PARÁMETRO C





DEMO TIME
(DEMO_4_SVM)



PREGUNTAS Y CONSULTAS